Your Scraping Robot

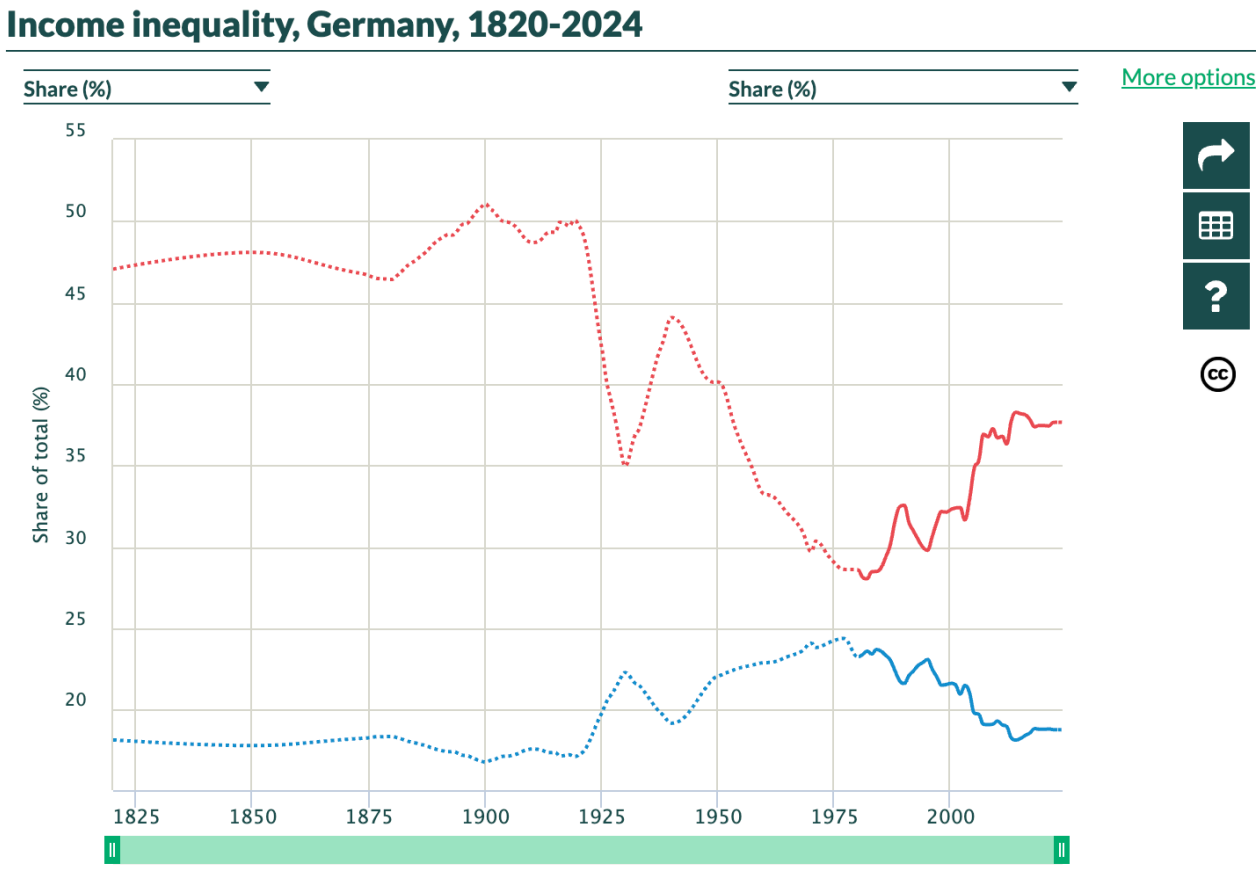
AUTHOR
Robert Gebeloff and Simon Wörpel, Dataharvest 2026

SCRAPING WITH A BROWSER EMULATOR

This is a class about advanced Web scraping techniques.

But before we begin, let's pause a moment and think about income inequality. The gap between the rich and the rest of us has been growing and growing in recent years, a global phenomenon that is at least partly to blame for many of the world's problems.

As journalists, we can tell this story in a macro way.



Source: World Inequality Database

Or we can tell the story in a micro way, showing how inequality plays out in a variety of different ways.

Late last year, my colleague Kurtis Lee came to me with an idea. He had noticed that blood plasma collection centers seemed to be appearing in surprising locations. These centers, which pay donors cash in exchange for donating blood components used in various medical therapies, have historically in the U.S. set up shop in poor neighborhoods, hoping to attract clients desperate for money.

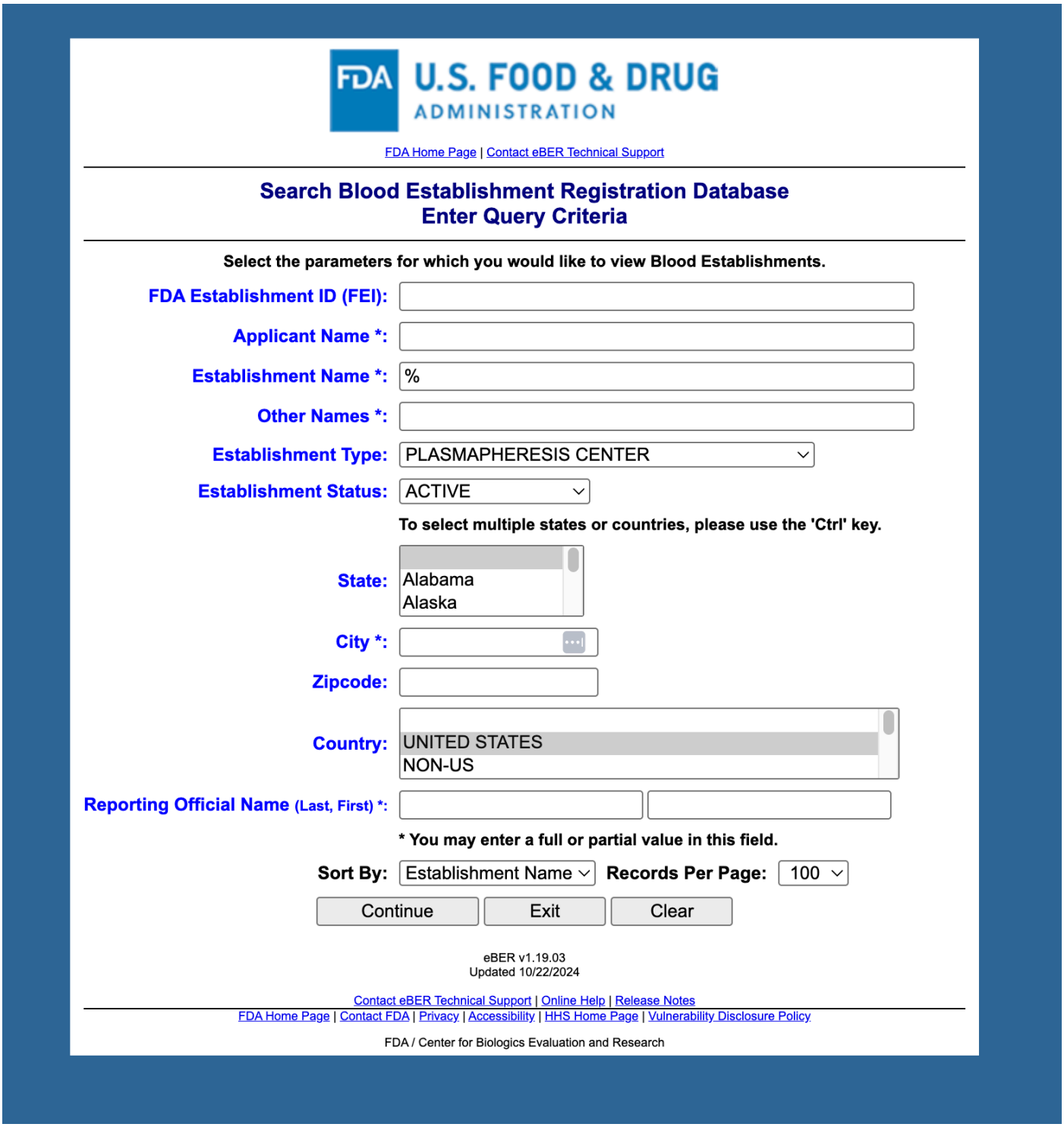
Kurtis suspected that these centers were increasingly moving into more suburban neighborhoods. Could I help him document this and figure out why?

We had some help. A couple of academics had gathered and studied information about the location of plasma centers a decade ago and had observed the beginnings of a shift out of poor neighborhoods. They were happy to share their data. But could we update their work and see what happened in the five years since they stopped collecting information?

The challenge was this: The U.S. Food and Drug Information maintains a database of blood plasma centers. But they don't make it available as open data, and filing a FOIA for the data would probably take months.

What the FDA does provide, however, is a way to look up plasma centers one at a time. And if that's technically possible, we felt we could write a script that would look them all up, one at a time.

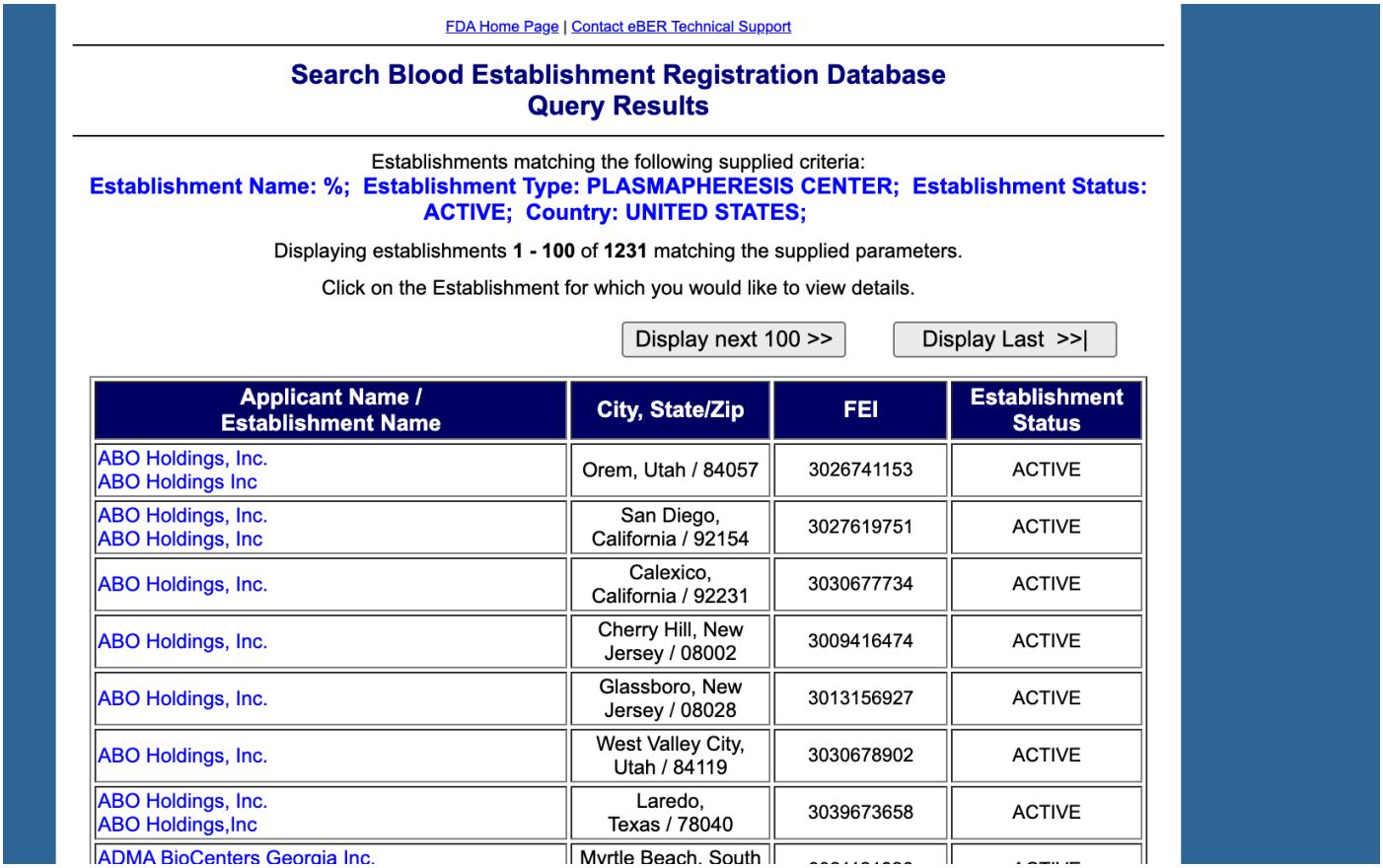
So we mapped out a plan. I would create a browser emulator that would simulate what a person would do if they had to look up every center one at a time.



Our emulator would go to the FDA search page and:

- Put a wildcard in the facility name page
- Select only Plasma Centers
- Select only Active facilities
- Display 100 results at a time.

It would then page through the results and harvest information about every facility,



FDA Home Page | Contact eBER Technical Support

Search Blood Establishment Registration Database
Query Results

Establishments matching the following supplied criteria:
Establishment Name: %; Establishment Type: PLASMAPHERESIS CENTER; Establishment Status: ACTIVE; Country: UNITED STATES;

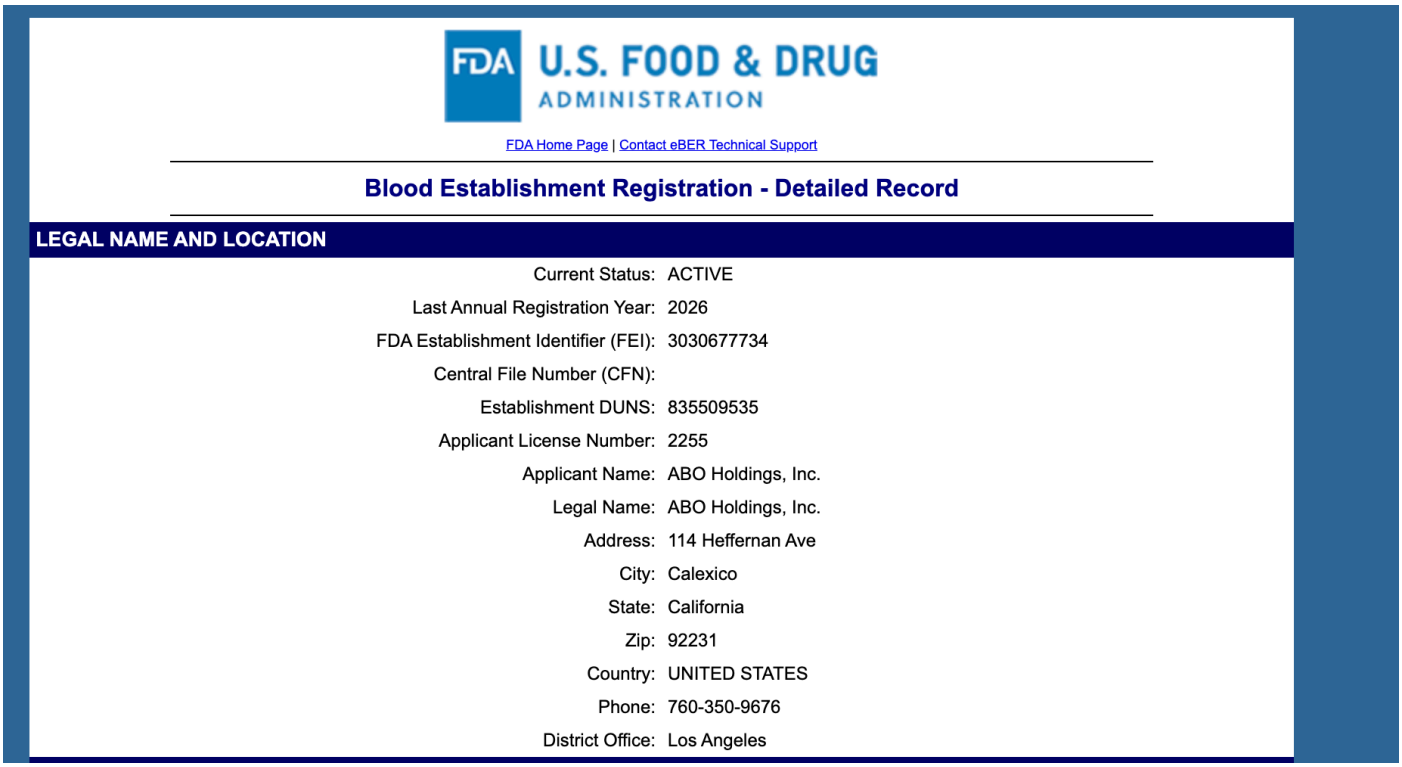
Displaying establishments **1 - 100** of **1231** matching the supplied parameters.

Click on the Establishment for which you would like to view details.

Display next 100 >> Display Last >>|

Applicant Name / Establishment Name	City, State/Zip	FEI	Establishment Status
ABO Holdings, Inc. ABO Holdings Inc	Orem, Utah / 84057	3026741153	ACTIVE
ABO Holdings, Inc. ABO Holdings, Inc	San Diego, California / 92154	3027619751	ACTIVE
ABO Holdings, Inc.	Calexico, California / 92231	3030677734	ACTIVE
ABO Holdings, Inc.	Cherry Hill, New Jersey / 08002	3009416474	ACTIVE
ABO Holdings, Inc.	Glassboro, New Jersey / 08028	3013156927	ACTIVE
ABO Holdings, Inc.	West Valley City, Utah / 84119	3030678902	ACTIVE
ABO Holdings, Inc. ABO Holdings,Inc	Laredo, Texas / 78040	3039673658	ACTIVE
ADMA BioCenters Georgia Inc.	Myrtle Beach, South		

including a URL that would lead to more information about each facility. It would then go through those URLS and harvest all of the data from those pages.



U.S. FOOD & DRUG ADMINISTRATION

FDA Home Page | Contact eBER Technical Support

Blood Establishment Registration - Detailed Record

LEGAL NAME AND LOCATION

Current Status:	ACTIVE
Last Annual Registration Year:	2026
FDA Establishment Identifier (FEI):	3030677734
Central File Number (CFN):	
Establishment DUNS:	835509535
Applicant License Number:	2255
Applicant Name:	ABO Holdings, Inc.
Legal Name:	ABO Holdings, Inc.
Address:	114 Heffernan Ave
City:	Calexico
State:	California
Zip:	92231
Country:	UNITED STATES
Phone:	760-350-9676
District Office:	Los Angeles

This is a job that can be done in several different ways and in several different languages. If you're a Python user, you'd probably use a library called Playwright to launch your emulator, but I'm going to

show you how I did this in R using a library called “chromote” – which as you might have guessed is shorthand for “Chrome Remote”.

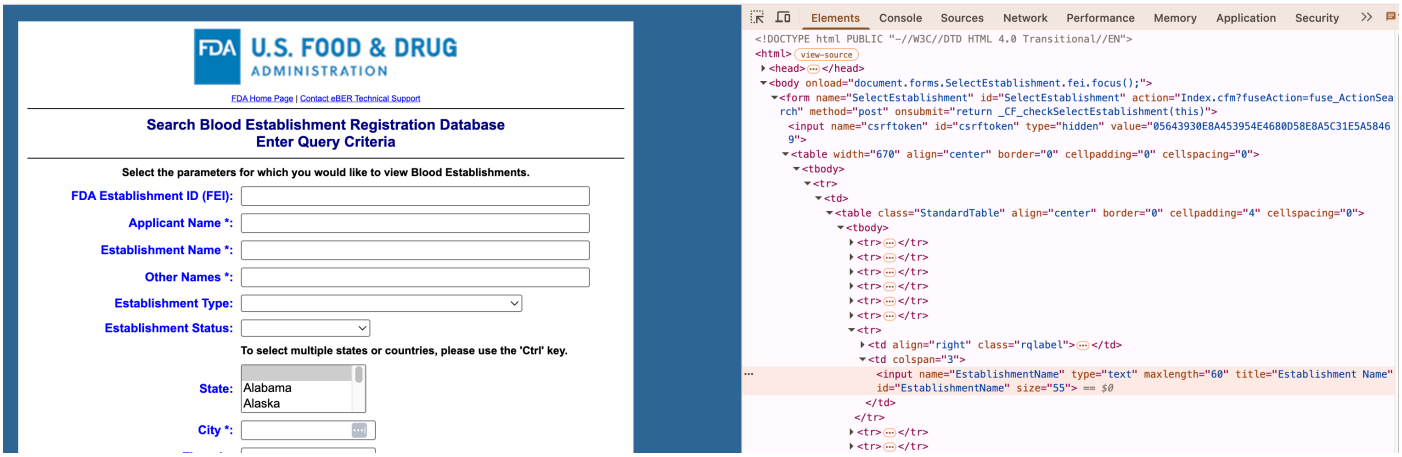
The idea is that your script will launch a browser, as if it were a human user, and then follow the steps a human user would do – if they had a lot of time and patience to click dozens of times and copy and paste lots of data.

Instead, our robotic entity will do all the clicking and data harvesting for us.

At this point, if you’d like to follow along on your own laptop, you are welcome to do so, but I am going to leave you with this entire script to try on your own time, so it might be easier to learn if, for now, you just watch this as a demonstration.

Feel free to interrupt at any time to ask questions.

Anyway, before I write the actual code, I need to do some pre-work. When I fill out the initial form on the FDA page, I’m going to need to know what those four form-fields we want to manipulate are called in the underlying code.



The four elements we’re going to deal with have the following names:

Establishment Name = Establishment, value = %

Establishment Status = EstablishmentStatus, value= ACTIVE

Establishment Type= EstablishmentType, value=3

Country = Country, value=US

Records Per Page=nrecords, value=100

Now we have to plan for how we’re going to launch a robot browser. This is a somewhat delicate subject because there are a lot of ways to do it and in my experience, as a journalist-trying-to-code, I’ve found that things that work one day tend to not work the next day.

So after doing much research on the topic, I’ve concluded that the best way to do this is launch a browser window outside of your script on a port that you define, and then have your script run through that port.

So on a Mac, you'd want to open your terminal and run this command:

```
/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome --remote-debugging-port=92
```

On a PC, it would be this:

```
"C:\Program Files\Google\Chrome\Application\chrome.exe" --remote-debugging-port=9222 --
user-data-dir="C:\chrome_debug_profile"
```

These two commands should work 95 percent of the time. If it doesn't work for you, you need to simply figure out the location of the Chrome executable – or the executable of some other Chromium-based browser (Edge should work, Firefox and Safari, probably not).

Ok, so by now we have our parameters figured out and our robot-browser fired up, let's code this up

Load the libraries we're going to use, including rvest, which is to help parse the HTML once we grab it.

```
library(tidyverse)
#install.packages("chromote")
library(chromote)
#install.packages("rvest")
library(rvest)
```

Establish a connection with our browser:

```
rc <- ChromeRemote$new(host = "localhost", port = 9222)
cm <- Chromote$new(browser = rc)
```

Open a tab in our new browser window and go to the FDA search page:

```
b <- cm$new_session()
b$go_to("https://www.accessdata.fda.gov/scripts/cber/CFAppsPub/")
```

Fill out the form. Here I'm using some javascript

```
b$Runtime$evaluate(expression = "
  var inputField = document.querySelector('input[name*=\"Establishment\"]');
  if (inputField) {
    inputField.value = '%';
  } else {
    console.error('Could not find the Establishment Name field.');
  }
")

b$Runtime$evaluate(expression = "
  var selectEl = document.querySelector('select[name*=\"EstablishmentStatus\"]');

  if (selectEl) {
```

```
    selectEl.value = 'ACTIVE';
    selectEl.dispatchEvent(new Event('change'));
    console.log('Successfully set dropdown to Active');
  } else {
    console.error('Could not find the EstablishmentStatus dropdown.');
  }
")

b$Runtime$evaluate(expression = "
  var selectC = document.querySelector('select[name*=\"Country\"]');

  if (selectC) {
    selectC.value = 'US';
    select.dispatchEvent(new Event('change'));
    console.log('Successfully set dropdown to Active');
  } else {
    console.error('Could not find the EstablishmentStatus dropdown.');
  }
")

b$Runtime$evaluate(expression = "
  var selectET = document.querySelector('select[name*=\"EstablishmentType\"]');

  if (selectET) {
    selectET.value = '3';
    select.dispatchEvent(new Event('change'));
    console.log('Successfully set dropdown to Active');
  } else {
    console.error('Could not find the EstablishmentStatus dropdown.');
  }
")

b$Runtime$evaluate(expression = "
  var selectN = document.querySelector('select[name*=\"nrecords\"]');

  if (selectN) {
    selectN.value = '100';
    selectEl.dispatchEvent(new Event('change'));
    console.log('Successfully set dropdown to Active');
  } else {
    console.error('Could not find the EstablishmentStatus dropdown.');
  }
")
```

Now that the form looks the way we want it, let's tell the robot to click the submit button:

```
b$Runtime$evaluate(expression = "
  var submitBtn = document.querySelector('[name=\"SubmitButton\"]');

  if (submitBtn) {
    submitBtn.click();
```

```
      console.log('SubmitButton successfully clicked!');
    } else {
      console.error('Could not find an element with name=\"SubmitButton\"');
    }
  ")

  # IMPORTANT: Pause R for a few seconds to let the search results load in Chrome

  Sys.sleep(3)
```

You'll notice that the results are paginated. So let's tell our robot that it should keep the script going as long as there is a button on the page with the id of "Display next".

```
# 1. Initialize an empty list and a tracking counter
all_pages_list <- list()
page_counter <- 1
has_next_page <- TRUE

message("Starting loop...")
```

Now let's write the loop:

```
while (has_next_page) {
  message(paste("Processing Page:", page_counter))

  # ------------------------------------------------------------
  # STEP 1: CAPTURE AND PARSE HTML (YOUR EXACT CODE)
  # ------------------------------------------------------------
  doc <- b$DOM$getDocument() # grab page and filter down to the html
  html_list <- b$DOM$getOuterHTML(nodeId = doc$root$nodeId)
  fda_html <- html_list$outerHTML
  parsed_page <- read_html(fda_html)

  #get the table with the relevent data
  raw_table <- parsed_page %>%
    html_element("table.tbl") %>%
    html_table()

  #get the first column as a list
  first_column_nodes <- parsed_page %>%
    html_elements("table.tbl td:first-child a")

  #pull out the urls from the first column as a list
  urls <- first_column_nodes %>%
    html_attr("href")

  #get the text from the first column as a list
  text <- first_column_nodes %>%
    html_text()
```

```
  #create a table out of those lists
  url_data <- tibble(
    text = text,
    url = urls
  )

  #create a final table by merging those two columns with the main table
  final_table <- raw_table %>%
    bind_cols(url_data)

  # Store this individual page's table inside our tracker list
  all_pages_list[[page_counter]] <- final_table
```

Now let's check to see if we have another page to process:

```
 # ------------------------------------------------------------
  # STEP 2: TARGET THE 'Display next' ID AND CLICK IT
  # ------------------------------------------------------------
  js_click_result <- b$Runtime$evaluate(expression = "
    var nextBtn = document.getElementById('Display next');
    if (nextBtn) {
      nextBtn.click();
      true;  // Tells R the button was found and clicked
    } else {
      false; // Tells R the button doesn't exist anymore
    }
  ")

  # Extract the true/false logical result from JavaScript
  button_was_clicked <- js_click_result$result$value
```

If there is another page to do, get ready to do it:

```
 # ------------------------------------------------------------
  # STEP 3: REPEAT OR BREAK THE LOOP
  # ------------------------------------------------------------
  if (button_was_clicked) {
    page_counter <- page_counter + 1

    # CRITICAL: Pause for 4 seconds to give Chrome time to pull
    # the next page down from the FDA servers before R tries to parse again
    Sys.sleep(4)
  } else {
    message("Finished! 'Display next' button is no longer on the page.")
    has_next_page <- FALSE
  }
 }
```

Finally, put all of the pages together into one table and write out results:

```
# ------------------------------------------------------------
# STEP 4: COMBINE ALL COLLECTED TABLES INTO A SINGLE DATA FRAME
# ------------------------------------------------------------
master_results_table <- bind_rows(all_pages_list)


#mutate the URL to add the prefix

final_table<-master_results_table%>%

mutate(full_url=paste("https://www.accessdata.fda.gov/scripts/cber/CFAppsPub/",url,sep=""
))

write.csv(final_table,"finaltable.csv")
```

Because we now have a URL for each entity, we can now go to those pages and harvest that data, including, most crucially for this project, the facility's address:

We are going to store this data into a blank table we're creating called detailed_entities:

```
# 1. Create an empty list to store the detailed data from each page
details_list <- list()

# 2. Get the total number of URLs to scrape for the progress monitor
total_urls <- nrow(final_table)

detailed_entities <- data.frame(category=character(),
                                value=character(),
                                facility_id=character())
```

Now we can iterate through all of our urls and capture the data. For the purposes of this class, I will do only the first 10:

```
message(paste("Starting deep scrape of", total_urls, "URLs..."))

#for (i in 1:total_urls) {
for (i in 1:10) {
  # Get the current URL
  current_url <- final_table$full_url[i]

  # Optional: Print progress so you know it hasn't frozen
  message(paste0("Scraping page ", i, " of ", total_urls, ": ", final_table$text[i]))

  # ------------------------------------------------------------
  # STEP 1: NAVIGATE TO THE DETAIL PAGE
  # ------------------------------------------------------------
  # Wrap this in a tryCatch in case a single link is broken, so it won't crash the whole
loop
  tryCatch({
    b$go_to(current_url)
```

```
    # ------------------------------------------------------------
    # STEP 2: PARSE THE CONTENT
    # ------------------------------------------------------------
    doc <- b$DOM$getDocument()
    html_list <- b$DOM$getOuterHTML(nodeId = doc$root$nodeId)
    page_html <- read_html(html_list$outerHTML)

    raw_table <- page_html %>%
      html_element("table.StandardTable")%>%
      html_table()

    selected_rows <- raw_table[c(6:19), ]%>%
      select(X1,X2)%>%
      rename(category=X1,value=X2)%>%
      mutate(facility_id=strsplit(current_url, "facility_id=")[[1]][2])

    detailed_entities<-detailed_entities%>%
      bind_rows(selected_rows)


  }, error = function(e) {
    message(paste("Error scraping row", i, ":", e$message))
    detailed_entities <- detailed_entities
  })

  # ------------------------------------------------------------
  # STEP 3: THE "POLITE" PAUSE (Crucial)
  # ------------------------------------------------------------
  # This generates a random pause between 2.5 and 5 seconds per page.
  # It breaks up the rhythmic typing pattern that firewalls look for.
  Sys.sleep(runif(1, min = 2.5, max = 5.0))
}

# ------------------------------------------------------------
# STEP 4: COMBINE AND MERGE BACK TO ORIGINAL DATA
# ------------------------------------------------------------
# Bind all the detail rows into one dataframe
all_details_table <- bind_rows(details_list)
```

In the end, after I scraped this data, we analyzed it and compared it to the older data provided by the academics, and my findings were a prominent part of the story.

So this all looks like a lot of detailed code and you might be wondering, will I ever write this code?

The answer is: No, but I didn't really write it either. Most of us these days learn to code by copying techniques we learn on the Internet or from AI.

In your case, what I hope is that you use my code as a launching pad to take on whatever project you are facing. My code probably won't work on your project perfectly right out of the box – but it will

hopefully get you started and you'll hopefully have learned enough from this session to be able to figure out how to modify the code and get it working.